

CI/CD with GitHub Actions

From “works on my machine” to shipping on every push — automated builds, tests, and deploys, all living inside your repo.

push

build

test

package

deploy

Prerequisites: you already know Linux commands and Docker. We build on both.

Six days, one repository

Everything lives in a single path-scoped monorepo — you clone once and grow it. Each day adds a folder and a workflow that only runs when its own folder changes.



DAY 1

First workflow

Events, jobs, steps, runners. A hello-world pipeline that runs on push.



DAY 2

CI for real

Build, test and lint a Node app on every pull request. Red X blocks merge.



DAY 3

Docker builds

Build an image in CI, run the container, push to GHCR.



DAY 4

Integration

Compose up a multi-service stack and test it end to end.



DAY 5

Deploy (CD)

Environments, secrets and an approval gate to a staging target.



DAY 6

Capstone

Wire the full pipeline for ShortLink — build, test, image, deploy.

PART ONE

Why CI/CD exists

Start with the pain. Every tool this week is an answer to a specific failure — name the failure first.



THE PAIN

“It worked on my machine”

Ten people branch off on Monday and merge on Friday. Nobody integrated in between. Now the branches conflict, half the tests were never run, and the build breaks — on deploy day.

-  Merges pile up all week — “integration hell” on Friday
-  Tests run manually, if at all — so they get skipped
-  The person who deploys is the only one who knows the steps
-  A broken main is discovered by users, not by the team

```
friday-4:55pm.sh

$ git merge feature-branch
CONFLICT (content): Merge conflict in app.js
Automatic merge failed; fix conflicts.

$ # did anyone run the tests?
$ npm test
Tests: 7 failed, 24 passed

$ # ship it anyway, it's late
$ scp -r ./dist prod:/var/www
$ ssh prod 'systemctl restart app'
• app.service: failed (exit 1)
$ # now what was step 3 again?
```

Continuous Integration

Every push merges into a shared branch and is automatically built and tested — many times a day, by a machine, the same way every time. Break something and you know in minutes, not on Friday.



Integrate often

Small changes merged daily keep conflicts tiny and reviewable — not a Friday pile-up.



Test automatically

The same build-and-test job runs on every push. No human decides whether to run it.



Fast feedback

A red check appears on the commit within minutes, so the author fixes it while it is fresh.

The test: if merging your branch could break the build and no machine would catch it, you are not doing CI yet.

Delivery vs. Deployment

Both start the same way: a green CI build produces a deployable artifact. The difference is the last step — who presses the button.



THE D IS DELIVERY

Continuous Delivery

Every green build is automatically packaged and pushed to staging, ready to go. Promoting to production is one manual click — a human gate. You could ship any commit; you choose when.

Safe default for most teams and where we start on Day 5.



THE D IS DEPLOYMENT

Continuous Deployment

No human gate. Every commit that passes CI goes all the way to production automatically. Requires deep trust in your test suite and fast rollback.

Powerful, but earn it — you need the automated confidence first.

Shared foundation: `commit` → `build` → `test` → `package` → `release`. **Only the final arrow differs.**

Anatomy of a pipeline

A pipeline is just stages that run in order. Each stage must pass before the next begins — the first failure stops the line.



Continuous Integration — build + test on every change

Continuous Delivery — package + deploy

PART TWO

GitHub Actions

The tool that runs your pipeline — no separate CI server, defined in YAML, living right next to your code.



No server to babysit



THE OLD WAY

A separate CI server

Stand up Jenkins on a box, patch it, secure it, keep plugins in sync, and hope it is up when you need to ship. Its config drifts from your code.



GITHUB ACTIONS

Built into the repo

GitHub already hosts your code, so the runners are one YAML file away. Nothing to provision. The pipeline is versioned with the project it builds.



Config as code

`.github/workflows/*.yaml` — reviewed in pull requests like any change.



Marketplace

Thousands of prebuilt actions: checkout, setup-node, docker login, deploy.



Hosted runners

Fresh Ubuntu, Windows or macOS VM per job — free minutes for public repos.



Event-driven

Triggers on push, PR, tag, schedule, or a manual button — not just commits.

Five words you need

Learn these five and every workflow file reads plainly. They nest inside each other, top to bottom.



Event

What starts it

a push, a pull request, a tag, a schedule, a manual click



Workflow

The whole pipeline

one .yaml file in .github/workflows/, triggered by events



Job

Runs on one runner

a fresh VM; jobs run in parallel unless you chain them with needs



Step

One task in a job

run a shell command, or use a prebuilt action



Action

A reusable step

packaged logic you call, e.g. actions/checkout@v4

Your first workflow, annotated



name

A human label shown in the Actions tab.



on

The event(s) that trigger this run.



jobs

One or more jobs; each gets a runner.



runs-on

Which OS image the job runs on.



steps

Ordered tasks: uses an action or runs a command.

```
.github/workflows/ci.yml

name: CI

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci
      - run: npm test
```

Events: the on: block

A workflow does nothing until an event fires. Pick the events that match the stage of the pipeline you are automating.



push

Code lands on a branch. The workhorse trigger for CI.



pull_request

Someone opens or updates a PR — test before merge, block the merge on red.



push: tags

A version tag like v1.2.0 — the classic trigger for a release/deploy.



schedule

Cron syntax. Nightly builds, dependency scans, cleanup jobs.



workflow_dispatch

A manual “Run workflow” button in the UI — your CD approval-friendly trigger.



workflow_call

Called by another workflow — reusable pipelines shared across repos.

Build, test, lint on every PR

Day 2's job. Three gates in one workflow — any red stops the merge. Notice the shared setup steps at the top.

```
● ● ● .github/workflows/day2-ci.yml

on: pull_request

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - name: Lint
        run: npm run lint
      - name: Unit tests
        run: npm test
```

What the reviewer sees

- ✓ Lint — passed
- ✓ Unit tests — 24 passed
- ✗ Build — failed on push

Merge is blocked until green.

Why this order

Setup once (checkout → node → install), then the cheap fast check (lint) before the slower one (tests). Fail-fast means a lint error never wastes time running the suite.

Matrix builds



THE PAIN

“Works on Node 20”

Your app has to run on Node 18 and 20, on Linux and macOS. Testing each by hand is four separate runs nobody bothers to do.



THE FIX

One matrix, N runs

Declare the axes once. Actions fans out a parallel job for every combination — four green checks from a five-line block.

```
strategy.matrix

jobs:
  test:
    strategy:
      matrix:
        node: [ 18, 20 ]
        os:   [ ubuntu, macos ]
    runs-on: ${{ matrix.os }}-latest
    steps:
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm test

# 2 nodes x 2 OSes = 4 parallel jobs
```

→ Same steps, every combination, all at once — no copy-pasted jobs.

A slow pipeline gets ignored

Every job starts on a blank VM — no dependencies, no layers. Re-downloading everything on each run is the number-one reason pipelines feel slow.

```
cache dependencies

- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ hashFiles(
      'package-lock.json') }}

# or, simpler — setup-node does it:
- uses: actions/setup-node@v4
  with: { cache: 'npm' }
```



Key on the lockfile

hashFiles(...) means the cache only rebuilds when dependencies actually change.



Layer cache for Docker

gha cache with Buildx reuses image layers so unchanged stages skip rebuilds.



Small, targeted jobs

Split unrelated work into parallel jobs instead of one long serial one.

Rule of thumb: if CI takes longer than a coffee break, people stop waiting for it — and start merging around it.

Docker in the pipeline

The Package stage, made real. CI builds your image, then pushes it to a registry — the exact image that passed tests is the one that ships.

```
build-and-push image

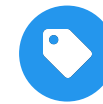
- uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${github.actor}
    password: ${secrets.GITHUB_TOKEN}

- uses: docker/build-push-action@v6
  with:
    push: true
    tags: ghcr.io/${github.repository}:${github.sha}
    cache-from: type=gha
    cache-to: type=gha,mode=max
```



GITHUB_TOKEN is free

GitHub injects a scoped token per run — no secret to create for pushing to GHCR.



Tag by commit SHA

github.sha makes every image uniquely traceable back to the exact commit.



type=gha layer cache

Unchanged Dockerfile stages are pulled from cache, not rebuilt — fast images.

Secrets, never in the file



NEVER DO THIS

Hard-coded keys

A token pasted into the YAML is committed to history forever and visible to anyone with read access. Rotating it means rewriting git history.



DO THIS

Encrypted secrets

Store them in repo or environment settings. They are encrypted, masked in logs, and referenced as `${{ secrets.NAME }}` at run time.

```
referencing a secret

- name: Deploy
  env:
    API_KEY: ${ secrets.API_KEY }
  run: ./deploy.sh

# in logs, the value prints as:
API_KEY=*** (auto-masked)
```



Environments add guardrails

- Scope secrets per environment — staging keys $\neq$ prod keys
- Required reviewers: a human must approve before the job runs
- Wait timers and branch restrictions on who can deploy
- *This is exactly how we build the delivery gate on Day 5*

CD: the deploy stage

Deploy is just another job — with two safety features: it needs the test job to pass first, and it targets a protected environment with an approval gate.

```
.github/workflows/day5-deploy.yml

jobs:
  test:
    runs-on: ubuntu-latest
    steps: [ ... npm ci, npm test ]

  deploy:
    needs: test          # gate: test must pass
    environment: staging # gate: approval + secrets
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ./deploy.sh
    env:
      TOKEN: ${ secrets.DEPLOY_TOKEN }
```



needs: test

The deploy job will not even start unless the test job finished green. Broken code can never reach the deploy step.



environment: staging

Because staging has a required reviewer, the run pauses here for a human click. That pause is continuous delivery.

Six practices to keep



Path filters

Scope each workflow to its own folder so a Day 2 fix never triggers Day 3's build.



Pin your actions

Use actions/checkout@v4, not @main — an unpinned action can change under you.



Least privilege

Set permissions: on the token. Grant only what the job needs, nothing more.



Fail fast, cheap first

Order steps lint → test → build so the quickest check catches errors soonest.



Small, named steps

One concern per step with a clear name. Green/red per step makes logs readable.



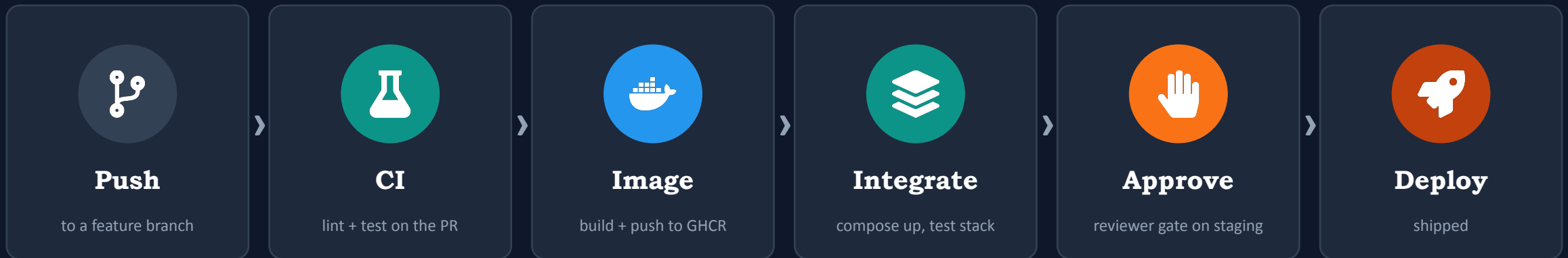
Secrets, always

No credential in the file. Reference `${{ secrets.X }}`; scope by environment.

THE CAPSTONE

ShortLink: the whole pipeline, wired

By Day 6 you point every concept from this deck at one real project — a URL shortener — and watch a commit travel from push to deployed, hands-off.



React · Node · Go · Python · Postgres · Redis — one Compose stack, one pipeline.

THE ONE IDEA

Let the machine do it — the same way, every time.

Manual integration is late, manual testing gets skipped, and manual deploys live in one person's head. CI/CD replaces all three with an automated, repeatable process that runs on every push. GitHub Actions is just how we write that process down.

Next class: Day 1 — clone the monorepo and write your first workflow. It runs on push and prints hello from a runner.

